MMA301 – MULTIPLATFORM MOBILE APP DEVELOPMENT

ASSIGNMENT 2

Integrated E-Commerce Mobile App

Remote Data • State • Firebase • Notification • Gemini AI • Debugging • Release

Nguyên tắc trọng tâm: Assignment không chỉ đánh giá sản phẩm chạy được. Sinh viên phải chứng minh chuỗi năng lực: hiểu yêu cầu → thiết kế có lý do → triển khai → kiểm thử → debug → giải thích → thích nghi khi yêu cầu thay đổi → sử dụng AI có trách nhiệm.

Thông tin	Nội dung
Assignment	Assignment 02 – Integrated React Native Application
Bối cảnh	E-Commerce Product App
CLO đánh giá chính	CLO4, CLO8–CLO13
CLO củng cố	CLO6–CLO7 (state/list) và CLO9 (data) được dùng lại trong integrated workflow
Hình thức	Bài có tính điểm; demo/interview/debug/transfer verification
Điểm rubric	10.0 điểm / 8 tiêu chí
Ngôn ngữ	JavaScript
Nền tảng	React Native + Expo theo cấu hình lớp
Assessment category	Assignment category = 10% học phần cho 2 assignments; điểm từng bài báo cáo thang 10, trọng số theo cấu hình lớp

CÁCH SỬ DỤNG TÀI LIỆU

Tài liệu này không phải đáp án. Hãy dùng như bản đồ triển khai và checklist chuẩn bị đánh giá. Mỗi giai đoạn mô tả mục tiêu, quyết định kỹ thuật, bằng chứng và câu hỏi sinh viên phải tự trả lời.

Giai đoạn	Việc cần làm	Bằng chứng nên có
1. Understand	Đọc đề, lập RTM, xác định screen/workflow/state/data/constraint.	RTM, workflow sketch, data/state sketch.
2. Design	Thiết kế navigation, component tree, state ownership, persistence/integration.	Design decisions + sơ đồ.
3. Build	Triển khai theo milestone; commit theo chức năng; tránh one-shot code dump.	Git history + chức năng chạy từng phần.
4. Verify	Test normal/negative/edge; ghi defect và retest.	Test matrix + evidence.
5. Explain	Tự giải thích purpose/input/trigger/transform/output/failure path.	Interview rehearsal checklist.
6. Transfer	Phân tích và xử lý change request nhỏ không dựa vào code mẫu.	Impact analysis + patch/test.
7. Defend	Demo, interview, debug/modification và AI evidence.	Bộ hồ sơ đánh giá hoàn chỉnh.

0. THÔNG TIN ASSIGNMENT VÀ CHÍNH SÁCH ĐÁNH GIÁ

Mục	Yêu cầu
Tên bài	Assignment 02 – Integrated E-Commerce Mobile App
Core flow	Catalog → Detail → Cart → persisted/business state
Remote data	Public API hoặc lecturer-provided backend
State	Context API hoặc Redux theo mức lớp
Firebase	Ít nhất 1 service/use case có nghiệp vụ thực
Notification	Business notification workflow + permission handling
Gemini/GenAI	Mandatory, gắn với business workflow
Debugging	React DevTools/log/debugger evidence
Release	Android APK/AAB hoặc lecturer-approved release evidence
Project name	StudentName_ClassCode hoặc quy ước lớp

0.1 Sản phẩm cần nộp

- Source code React Native có thể cài dependency và chạy lại.
- README: business workflow, setup/config (không chứa secret), state/data architecture, Firebase/notification/Gemini setup, release steps, limitations, AI usage.
- RTM + Design Decision Record + integration map.
- Test matrix + tối thiểu 2 debug logs.
- Release artifact hoặc build evidence theo yêu cầu lớp.
- AI usage log nếu dùng AI hỗ trợ; Git history nếu lớp dùng Git.

0.2 Chính sách trung thực học thuật và AI

- Sinh viên chịu trách nhiệm với toàn bộ source code, cấu hình và tài nguyên nộp; phải giải thích được phần mình sử dụng.
- Được phép dùng slide, tài liệu chính thức, ví dụ đã học và AI assistant theo quy định của lớp.
- Nếu dùng AI ở mức đáng kể: ghi mục đích/prompt hoặc tham chiếu, phần đã áp dụng, phần đã sửa và cách kiểm chứng.
- Không nộp source trọn bộ do AI/người khác tạo mà không hiểu; code chạy nhưng không giải thích hoặc không sửa được yêu cầu nhỏ sẽ bị phản ánh ở rubric tương ứng.
- Không commit secret/API key thật vào repository hoặc file nộp.

0.3 Quy trình buổi đánh giá – mô hình khuyến nghị

Pha	Sinh viên thực hiện	Giảng viên quan sát/chấm
A. Identity & Run	Mở đúng bản nộp, chạy app bằng lệnh chuẩn, demo nhanh workflow chính.	Tính xác thực; môi trường; project chạy.
B. Requirement Demo	Demo từng requirement theo RTM, gồm cả failure/empty/edge state.	Problem Understanding + Implementation.
C. Code Interview	Mở file do giảng viên chỉ định và giải thích state/data/event flow.	Code Explanation + Design Reasoning.
D. Verification / Debug	Chạy test case hoặc phân tích một lỗi mới.	Testing + Debugging.
E. Transfer Task	Phân tích impact và thực hiện/mô tả một thay đổi nhỏ.	Modification / Transfer.
F. AI Evidence	Trình bày AI log nếu có và cách kiểm chứng output.	AI Usage & Engineering Practice.

Phạm vi: Assignment 2 là bài tích hợp. Không tăng điểm chỉ bằng feature count; ưu tiên workflow end-to-end, integration đúng, failure handling, debug và khả năng giải thích.

1. TÓM TẮT YÊU CẦU VÀ PHẠM VI THỰC HIỆN

1.1 Business workflow bắt buộc

1. Startup → restore cart/favorites/preference/session-like state theo design → load product catalog từ remote source.
2. Browse/Search → FlatList pagination/infinite scrolling + refresh → loading/empty/error/retry rõ ràng.
3. Product Detail → Add/Update/Remove Cart → tổng tiền và item count tính đúng.
4. Firebase → lưu ít nhất một business state (wishlist/order history/cart sync/preference hoặc use case được duyệt).
5. Gemini/GenAI → nhận context thực từ catalog/user need → tạo output có ích → app validate/match output trước khi action/display.
6. Notification → một business event chứng minh permission + notification flow theo cấu hình lớp.
7. Failure: network/Firebase/AI/permission error không làm app crash; người dùng có feedback hợp lý.
8. Release-like Android build được tạo và chạy ngoài development-only workflow.

1.2 Minimum screens/modules

Screen/module	Trách nhiệm
ProductListScreen	Catalog, search/filter, FlatList, pagination/infinite, refresh, loading/empty/error.
ProductDetailScreen	Thông tin sản phẩm + add to cart + optional AI entry point.
CartScreen	Items, quantity, remove, derived totals, persistence.
Saved/Orders/Wishlist	Firebase-backed business state hoặc tương đương được duyệt.
AI Recommendation/Compare UI	Gemini feature gắn với catalog/business context.
Notification/Settings area	Permission/status/test flow hoặc tương đương.

1.3 Requirement Traceability Matrix tối thiểu

ID	Yêu cầu	Bằng chứng UI/code	Test tối thiểu
R01	Remote product data	service/API	success/error
R02	FlatList + pagination/infinite	ProductList	load more/end
R03	Search/filter + refresh	ProductList	hit/miss/refresh
R04	Product detail	Detail	valid id/missing
R05	Cart state	hook/context/redux	add/update/remove
R06	Derived totals	cart selector/logic	0/1/many items
R07	Persistence	AsyncStorage/SQLite as used	restart
R08	Firebase use case	Firebase service	write/read/error
R09	Notification flow	notification service	permission allow/deny
R10	Gemini AI integration	AI service/UI	valid/error/invalid output
R11	Debug evidence	debug log/tool	root cause + retest
R12	Release build	APK/AAB/build evidence	install/run

1.4 Phạm vi và giới hạn

- Không bắt buộc payment gateway thật.
- Không bắt buộc publish lên store công khai nếu lớp chỉ yêu cầu build evidence.
- Không bắt buộc nhiều Firebase services; một use case có nghiệp vụ rõ tốt hơn nhiều integration trang trí.
- Gemini generic chatbot không liên quan catalog/business không đạt yêu cầu AI integration.
- TypeScript không bắt buộc.

2. HƯỚNG DẪN PHÂN TÍCH YÊU CẦU – PROBLEM UNDERSTANDING

Workflow	Input	Process	Output / failure
Catalog	query/page/remote data	fetch → normalize → render	list / empty / error / retry
Cart	product, quantity	add/update/remove → derive total	cart UI + persisted state
Firebase	business record/user action	write/read/sync	saved state / permission/network error
Gemini	user need + catalog context	prompt/request → validate/match	recommendation/actionable UI / fallback
Notification	event + permission	request/register/schedule/receive	notification/status / denied
Release	config/assets/build profile	build/install/run	release artifact / build failure

2.2 Worksheet – điền trước khi code

- Source of truth của product/cart/wishlist là gì?
- State nào local/shared/persistent/remote?
- Pagination API thật hay client-side? end-of-list xác định thế nào?
- Firebase use case giải quyết vấn đề gì, collection/record tối thiểu là gì?
- Gemini nhận context nào? output nào cần validate/match với catalog?
- Notification gắn với business event nào? Nếu permission denied thì UI ra sao?
- Secret/config nào không được commit?
- Nếu API/Firebase/Gemini lỗi đồng thời, phần nào của app vẫn phải dùng được?

3. HƯỚNG DẪN THIẾT KẾ GIẢI PHÁP – SOLUTION DESIGN & REASONING

Screens → hooks/state → services → Remote API / Firebase / AI / Notification
Persistent local state → AsyncStorage/SQLite (theo design)
UI luôn biểu diễn loading / data / empty / error / permission states

3.1 Separation of concerns

Layer	Ví dụ	Không nên
Screen/UI	render + event wiring	chứa toàn bộ fetch/Firebase/Gemini logic
Hook/state	useCart/useProducts/shared store	mix unrelated state
Service	productService/firebaseService/aiService	hard-code UI/state vào service
Persistence	storage helper	rải key/JSON parse ở nhiều screen
Config	environment/config module	commit secret vào source

3.2 Design Decision Record – tối thiểu 6 quyết định

Quyết định	Lựa chọn của tôi	Lý do	Impact nếu đổi requirement
State management			
API/service structure			
Pagination strategy			
Persistence			
Firebase use case			
Gemini context/validation			
Notification flow			

4. HƯỚNG DẪN TRIỂN KHAI THEO MILESTONE – IMPLEMENTATION

Milestone	Gate đạt	Cách kiểm tra	Evidence
M1	Navigation + UI skeleton	screens mở đúng	UI + Git
M2	Remote API + ProductList	loading/data/error + pagination	API evidence
M3	Detail + Cart/state	add/update/remove + totals	cart tests
M4	Persistence + refresh/search	restart + edge state	storage evidence
M5	Firebase business use case	write/read/error	Firebase evidence
M6	Notification workflow	permission + business event	notification evidence
M7	Gemini feature	context → response → validation → UI	AI integration evidence
M8	Debug/integration hardening	2 bugs root-cause + retest	debug logs
M9	Release candidate	build/install/run + README	release artifact

Nguyên tắc: Không cần triển khai mọi thứ cùng lúc. Mỗi milestone phải tạo một vertical slice có thể chạy/kiểm chứng trước khi mở rộng.

5. HƯỚNG DẪN KIỂM THỬ – TESTING & VERIFICATION

#	Nhóm	Case	Thao tác	Kết quả mong đợi
1	API	Success	load catalog	list hiển thị
2	API	Network error	simulate/offline	error + retry
3	List	Empty	no products	empty state
4	Pagination	End	scroll cuối	không duplicate/infinite request
5	Search	Hit/Miss	keyword	đúng/no-result
6	Cart	Add same item	add 2 lần	quantity/total đúng
7	Cart	Quantity 0/remove	decrease/remove	state đúng
8	Persistence	Restart	cart/favorite exists	restore theo design
9	Firebase	Write/Read	save item/order	data phản ánh đúng
10	Firebase	Error	permission/network	feedback/fallback
11	Notification	Denied	deny permission	app không crash; trạng thái rõ
12	Notification	Allowed/event	trigger workflow	notification/status đúng
13	Gemini	Valid	business prompt/context	output hợp lý + mapped
14	Gemini	Invalid/error	bad/timeout response	fallback; không trust ID giả
15	Release	Install/run	release build	core workflow chạy

6. HƯỚNG DẪN DEBUG – DEBUGGING & PROBLEM SOLVING

Quy trình 5 bước: (1) mô tả triệu chứng quan sát được; (2) khoanh vùng screen/component/state/service; (3) kiểm tra input/state; (4) kiểm tra điểm biến đổi/async call; (5) sửa nhỏ nhất → chạy lại case → kiểm tra regression.

Triệu chứng	Điểm kiểm tra đầu tiên	Câu hỏi dẫn đường
List gọi API lặp	useEffect/dependencies/onEndReached	dependency hoặc guard loading/end đúng?
Cart total sai	derived state/reducer	quantity/price type? compute ở đâu?
Firebase write không thấy	config/auth/rules/network	promise error/log/rule?
Notification không nhận	permission/token/config/app state	permission granted? handler/register?
Gemini trả product không tồn tại	context/output validation	có match với catalog thực? fallback?
Release chạy khác dev	env/config/build profile	biến môi trường/package/plugin nào khác?

6.2 Debug Log – tối thiểu 2 lỗi thật

Lỗi	Giả thuyết	Cách kiểm tra	Root cause	Fix	Retest

Trong buổi chấm: Nếu giảng viên đưa ra bug mới, hãy nói thành tiếng: triệu chứng → giả thuyết → nơi kiểm tra → bằng chứng → sửa → retest.

7. HƯỚNG DẪN CHUẨN BỊ GIẢI THÍCH CODE – CODE EXPLANATION

Sinh viên phải mở được code của mình và giải thích theo luồng thực thi; không nên chỉ trả lời định nghĩa thuộc lòng.

Tầng	Cách giải thích

Tầng	Cách giải thích
1. Purpose	Screen/component/hook/service này giải quyết trách nhiệm gì?
2. Input	Props/state/data/parameter nào đi vào?
3. Trigger	Event/useEffect/navigation/user action nào kích hoạt?
4. Transform	Logic nào validate/filter/map/call API/update state?
5. Output	State/UI/navigation nào thay đổi? Failure path ra sao?

7.2 Chủ đề phải giải thích được

- Product data flow từ API → service/hook → state → FlatList.
- Pagination/loading guard và tránh duplicate request.
- Cart state + derived totals + persistence.
- Firebase use case và failure handling.
- Notification permission/event/handler flow.
- Gemini input/context/prompt → request → response → validation → UI.
- Config/secret handling ở mức assignment.
- Debugging evidence và khác biệt dev/release.

7.3 "Mở file và chỉ code" – rehearsal checklist

Yêu cầu rehearsal	Tôi làm được trong ≤30 giây?
Mở product service + hook/fetch logic	□
Mở pagination/onEndReached guard	□
Mở cart state/useCart/reducer	□
Mở Firebase service và chỉ business record	□
Mở notification permission/handler	□
Mở AI service + response validation	□
Mở build/config không chứa secret	□

8. HƯỚNG DẪN XỬ LÝ YÊU CẦU THAY ĐỔI – MODIFICATION / TRANSFER TASK

Mục tiêu: chứng minh sinh viên không chỉ nhớ code hiện tại mà có thể chuyển kiến thức sang requirement mới. Trước khi code, hãy phân tích impact.

8.1 Bài luyện mức cơ bản

- Thêm sort theo price/rating.
- Thêm retry button cho API error.
- Thêm field note cho cart item.
- Đổi validation của quantity.

8.2 Bài luyện mức trung bình

- Thêm category filter kết hợp search.
- Chuyển wishlist từ local sang Firebase hoặc ngược lại.
- Thêm notification khi favorite item thay đổi.
- Đổi Gemini use case từ recommendation sang comparison giữ cùng catalog data.

8.3 Bài luyện nâng cao

- Thêm offline cache + stale indicator.
- Thay pagination page-number bằng cursor/mock strategy.
- Thêm structured JSON response cho Gemini và validate schema trước khi render.

8.4 Impact Analysis Worksheet

Change request	Screen/component	State/data	Service/storage	UI/permission	Tests phải sửa/thêm

9. HƯỚNG DẪN SỬ DỤNG AI CÓ KIỂM SOÁT – AI USAGE & ENGINEERING PRACTICE

9.1 AI có thể hỗ trợ

- Giải thích concept hoặc đoạn code do sinh viên đang làm.
- Gợi ý component/state/data structure hoặc refactoring.
- Đưa giả thuyết debug và checklist kiểm tra.
- Sinh test case/edge case để sinh viên chọn và kiểm chứng.
- Review naming, UX/accessibility, validation, README hoặc design alternatives.

9.2 AI không được thay thế năng lực

- Không dùng AI để tạo toàn bộ assignment rồi nộp như sản phẩm của mình.
- Không coi output AI là đúng nếu chưa chạy/test/đối chiếu requirement.
- Không dùng AI để che giấu việc không giải thích được state/event/data flow.
- Trong buổi đánh giá trực tiếp, lecturer có thể yêu cầu modification/debug không dùng AI.

9.3 Quy trình AI → Verify → Adopt

9. Ask – đặt câu hỏi cụ thể.
10. Inspect – đọc output và xác định giả định/rủi ro.
11. Adapt – chỉnh cho project của mình.
12. Run – chạy đúng test case.
13. Explain – tự giải thích được logic và side effect.
14. Record – ghi prompt/reference + phần đã đổi + cách kiểm chứng.

9.4 AI Usage Log tối thiểu

#	Task / Prompt / Ref	Mục đích	Output dùng?	Tôi đã sửa gì?	Tôi kiểm chứng thế nào?

10. RUBRIC ĐÁNH GIÁ – 10.0 ĐIỂM

Rubric đánh giá năng lực thực chất, không chỉ đánh giá việc sản phẩm "có chạy". Implementation tối đa 2.0/10; 8.0 điểm còn lại đến từ hiểu bài toán, thiết kế, kiểm chứng, debug, giải thích, transfer và AI engineering practice.

#	Tiêu chí	Điểm tối đa	Bằng chứng chính
1	Problem Understanding	1.0	RTM + business workflow + failure/security constraints.
2	Solution Design & Reasoning	1.5	State/service/integration design + decision record.
3	Implementation	2.0	API/cart/Firebase/notification/Gemini/release core workflow.
4	Testing & Verification	1.5	15-case matrix normal/negative/edge + actual evidence.
5	Debugging & Problem Solving	1.5	2 debug logs + live root-cause reasoning.

#	Tiêu chí	Điểm tối đa	Bằng chứng chính
6	Code Explanation	1.0	Trace API/cart/Firebase/notification/AI flow trong source.
7	Modification / Transfer Task	1.0	Change request + impact analysis + retest.
8	AI Usage & Engineering Practice	0.5	AI-assistance log + verify/adapt/explain.

10.1 Mức đánh giá chi tiết

Tiêu chí	Excellent / Tối đa	Good / Khá	Fair / Trung bình	Poor / Chưa đạt
Problem Understanding – 1.0	0.9–1.0: Nêu đúng scope, workflow, constraint, input/output; RTM khớp yêu cầu và test.	0.7–0.8: Hiểu phần lớn; thiếu chi tiết nhỏ nhưng không sai core requirement.	0.4–0.6: Hiểu rời rạc; bỏ sót constraint hoặc không rõ input/output/failure state.	0–0.3: Không mô tả được bài toán; nhầm core requirement hoặc chỉ dựa vào code có sẵn.
Solution Design & Reasoning – 1.5	1.35–1.50: Navigation/component/state/data thiết kế hợp lý; nêu trade-off và impact khi đổi yêu cầu.	1.05–1.30: Thiết kế hợp lý, giải thích được phần chính; một số quyết định còn theo thói quen.	0.60–1.00: Cấu trúc chạy được nhưng ownership/coupling chưa rõ; khó giải thích decision.	0–0.55: Thiết kế chắp vá; logic rải rác; không biết vì sao đặt state/data/logic như vậy.
Implementation – 2.0	1.80–2.00: Core workflow đầy đủ, tích hợp ổn định, state/data update đúng, failure state hợp lý.	1.40–1.75: Core functions hầu hết chạy; có lỗi nhỏ hoặc chất lượng chưa đều.	0.80–1.35: Chạy một phần; thiếu/mất ổn định ở workflow chính; code khó duy trì.	0–0.75: Project không chạy hoặc thiếu nhiều core requirements; không làm chủ source.
Testing & Verification – 1.5	1.35–1.50: Test matrix đủ normal/negative/edge; expected/actual rõ; bug fix có retest.	1.05–1.30: Test core functions và một số edge case; evidence tương đối rõ.	0.60–1.00: Test chủ yếu happy path; thiếu edge/actual evidence.	0–0.55: Hầu như không có test có cấu trúc; chỉ demo ngẫu nhiên.
Debugging & Problem Solving – 1.5	1.35–1.50: Khoanh vùng có hệ thống, dùng log/DevTools/debugger hợp lý, tìm root cause và retest.	1.05–1.30: Sửa được lỗi phổ biến, giải thích nguyên nhân tương đối rõ.	0.60–1.00: Thử-sai nhiều; cần gợi ý; sửa được nhưng chưa rõ root cause.	0–0.55: Không biết bắt đầu debug; phụ thuộc người khác/AI để sửa.
Code Explanation – 1.0	0.9–1.0: Mở đúng file và giải thích purpose-input-trigger-transform-output/failure path; hiểu state/data flow.	0.7–0.8: Giải thích được phần lớn code mình dùng; còn một số chi tiết chưa sâu.	0.4–0.6: Giải thích cú pháp cơ bản nhưng không theo được luồng dữ liệu/thực thi.	0–0.3: Không giải thích được code quan trọng hoặc câu trả lời mâu thuẫn với source.
Modification / Transfer Task – 1.0	0.9–1.0: Phân tích impact, sửa change request đúng, kiểm thử lại và giải thích thay đổi.	0.7–0.8: Thực hiện được thay đổi với ít gợi ý; test cơ bản.	0.4–0.6: Cần nhiều gợi ý; sửa được một phần hoặc gây regression.	0–0.3: Không thể xử lý thay đổi nhỏ nếu không có mẫu sẵn.
AI Usage & Engineering Practice – 0.5	0.45–0.50: Dùng AI minh bạch, có mục tiêu, tự kiểm chứng/chỉnh sửa, giải thích được kết quả.	0.35–0.40: Dùng AI hợp lệ và có kiểm chứng nhưng log/giải trình chưa đầy đủ.	0.20–0.30: Có dùng AI nhưng kiểm chứng hạn chế; phụ thuộc gợi ý khá nhiều.	0–0.15: Không minh bạch hoặc phụ thuộc source/AI trọn bộ mà không hiểu.

10.2 Scorecard sinh viên tự chấm trước đánh giá

Tiêu chí	Max	Tự chấm	Evidence tôi sẽ mở
Problem Understanding	1.0		
Solution Design & Reasoning	1.5		
Implementation	2.0		
Testing & Verification	1.5		
Debugging & Problem Solving	1.5		
Code Explanation	1.0		
Modification / Transfer Task	1.0		
AI Usage & Engineering Practice	0.5		
TỔNG	10.0		

11. CHECKLIST TRƯỚC KHI NỘP VÀ TRƯỚC BUỔI ĐÁNH GIÁ

11.1 Checklist yêu cầu bắt buộc

- □ Catalog remote data + pagination/search/refresh.
- □ Product Detail + Cart + totals.
- □ Persistence theo design.
- □ Firebase business use case.
- □ Notification permission/business flow.
- □ Gemini feature gắn business context.
- □ Release build evidence.

11.2 Checklist chất lượng kỹ thuật

- □ Loading/empty/error/retry rõ ràng.
- □ Không duplicate pagination request.
- □ Không commit secret/API key.
- □ Firebase/AI errors có handling.
- □ 2 debug logs có root cause/retest.
- □ README chạy/config/release rõ.

11.3 Checklist sẵn sàng phỏng vấn

- □ Trace được 3 flow: API→List, Cart→Persistence, Gemini→Validation→UI.
- □ Mở đúng service/hook/config trong 30 giây.
- □ Giải thích Firebase/notification failure path.
- □ Đã luyện 1 modification task.

11.4 Checklist hồ sơ đánh giá

- □ Source/zip/repo đúng bản.
- □ README final.
- □ RTM + Design Decision Record.
- □ Test matrix final.
- □ 2 debug logs.
- □ Release artifact/evidence.
- □ AI usage log nếu có.

11.5 Kế hoạch chuẩn bị 45–60 phút trước đánh giá

Thời gian	Việc làm
0–10 phút	Mở/clone đúng bản nộp; cài dependency; chạy app; xác nhận đúng version.
10–25 phút	Chạy smoke test toàn bộ core workflow + negative/edge cases chính.
25–35 phút	Mở RTM/design/state/data sketch; luyện giải thích 2–3 luồng chính.
35–45 phút	Luyện 1 debug case và 1 modification case không dùng AI.
45–60 phút	Kiểm tra README/test matrix/debug log/AI log/evidence; chuẩn bị demo trạng thái ổn định.

12. PHỤ LỤC – CÂU HỎI INTERVIEW / TỰ LUYỆN

12.1 Data / list / state

- onEndReached có thể gọi lặp khi nào?
- Source data và displayed data khác nhau thế nào?
- Cart total nên lưu hay derive? Vì sao?

12.2 Firebase / notification

- Firebase feature giải quyết business problem gì?
- Nếu permission notification bị deny thì workflow thay đổi ra sao?
- Nếu Firebase offline/error thì UI thể hiện gì?

12.3 Gemini AI

- Prompt/context lấy từ đâu?
- Output nào phải validate trước khi dùng?
- Nếu AI hallucinate product ID/name thì phát hiện thế nào?
- Credential/security approach của bạn là gì?

12.4 Debug / release / transfer

- Dev chạy nhưng release lỗi: khoanh vùng gì?
- Thêm category filter: state/service/UI/test nào bị ảnh hưởng?

12.5 AI-assisted coding

- Bạn dùng AI để viết/debug phần nào?
- Output nào bạn từ chối và vì sao?
- Bạn đã kiểm chứng output bằng test nào?

12.6 Final self-declaration

Sinh viên tự xác nhận: “Tôi đã tự kiểm tra assignment, có thể giải thích các phần chính, có thể phân tích/sửa một thay đổi nhỏ và chịu trách nhiệm với mọi code, tài nguyên và AI output tôi sử dụng.”